

Propuesta de Tesina para la obtención del grado Licenciado en Ciencias de la Computación

10 de junio de 2020

Postulante: Denise Marzorati

Director: Ernesto Kofman

Codirector: Joaquín Fernández

1. Situación del postulante

Al momento de presentar la propuesta, el estudiante se encuentra realizando el trabajo final de la materia Compiladores.

Se espera dedicar al menos veinte horas semanales a la realización de la tesina.

2. Título

Causalización eficiente de grandes sistemas de ecuaciones algebraico-diferenciales

3. Estado del arte

3.1. Motivación

En el mundo real existen sistemas, cuyo comportamiento es difícil de evaluar, ya sea porque es muy costoso (por ejemplo, un puente siendo derribado), representa algún tipo de peligro para los actores implicados (un reactor nuclear explotando), etc.

Es debido a eso se ha desarrollado la Teoría General de Sistemas, que estudia las propiedades generales de los sistemas dinámicos y sus respectivos modelos. En primera instancia se desea representar mediante algún modelo (matemático) el sistema en cuestión. Luego, a partir del modelo, se desea simular el comportamiento del sistema original, experimentando con el modelo construido.

Dado que modelar y simular es de gran utilidad, constantemente se busca poder aplicar estas técnicas a sistemas cada vez más complejos. Al intentar hacerlo surgen nuevos desafíos; este trabajo en particular pretende investigar y contribuir al modelado y simulación de sistemas de gran escala.

3.2. Ecuaciones diferenciales

Los sistemas dinámicos continuos suelen ser representados mediante sistemas de ecuaciones diferenciales algebraicas (EDA), que están compuestos por ecuaciones diferenciales y ecuaciones algebraicas, o son equivalentes a un sistema de este tipo. Dichas ecuaciones son de la forma:

$$F(\dot{x}, x, t) = 0$$

donde $x(t)$ es un vector de variables dependientes (vector de estados), $\dot{x}(t)$ la derivada con respecto al tiempo, y t es el tiempo.

La resolución de este tipo de ecuaciones es difícil [7]; por ese motivo, se suele convertir el sistema de EDAs, a un sistema de EDOs (ecuaciones diferenciales ordinarias), que se puede resolver con métodos numéricos más simples y robustos que aquellos usados para EDAs [3]. Las EDOs son de la forma:

$$\dot{x} = f(x, t)$$

3.3. Causalización

El mencionado proceso de convertir EDAs a EDOs se denomina *causalización*. Para resolver el sistema de ecuaciones es necesario ordenarlas, de manera que se puedan despejar las incógnitas. Dado que muchos de estos sistemas presentan *lazos algebraicos* y/o *singularidades estructurales* se recurre a la teoría de grafos, para:

1. Determinar de qué ecuación se despejará cada incógnita. Para ello se construye un grafo bipartito, cuyo primer conjunto de vértices será conformado por las ecuaciones, el otro conjunto por las incógnitas, y los arcos representarán la presencia de una incógnita en una ecuación. Luego, se empareja incógnitas con ecuaciones encontrando un matching máximo.
 - a) En caso de no poder hallar dicho matching, se dice que el sistema posee singularidades estructurales. Para resolver este problema se aplica un algoritmo de reducción de índices (Pantelides [6]) iterativamente hasta que dichas singularidades desaparezcan. La principal idea de estos algoritmos es derivar selectivamente algunas de las ecuaciones del sistema.
2. Ordenar los apareamientos de manera tal que se sepa qué incógnita debe despejarse primero. Los sistemas que presentan lazos algebraicos introducen la necesidad de despejar varias incógnitas al mismo tiempo. Por eso, luego de encontrar el matching, se construye un nuevo grafo sobre el cual se aplica el algoritmo de Tarjan [8], que devuelve un orden topológico que describe en qué orden se deben resolver las ecuaciones, y también retorna las *componentes fuertemente conexas* del grafo, que representan conjuntos de ecuaciones que deben ser resueltas simultáneamente (lazos algebraicos).
3. Para resolver los lazos algebraicos se suele recurrir a un algoritmo de *tearing*. La idea básica de estos algoritmos es: se asume conocida una de las variables del lazo, se elimina una de las ecuaciones que involucra

dicha variable, y se ordenan el resto de las ecuaciones del lazo. Si este procedimiento conduce a un sistema sin lazo algebraico, el lazo original debe resolverse entonces sobre la variable que se asumió conocida.

3.4. Modelica

Modelica [5] es un lenguaje no propietario, orientado a objetos y basado en ecuaciones, desarrollado a partir del paradigma de modelado acausal, para el modelado de sistemas continuos y discretos.

El código de dicho lenguaje debe convertirse a algún lenguaje de programación para su ejecución; dicha tarea que es realizada por compiladores de Modelica (como por ejemplo: MapleSim, OpenModelica, Dymola, etc.). Un grupo CIFASIS-CONICET y FCEIA-UNR ha desarrollado el compilador de Modelica, *ModelicaCC*, sobre el cual se desea volcar los resultados que se obtengan en el futuro trabajo.

3.5. Conexiones en Modelica

Previo a la etapa de causalización, se *aplana* el sistema recibido, para simplificarlo. Modelica provee varias construcciones y estructuras que facilitan el trabajo del programador, pero que luego deben ser reemplazadas por sus componentes más básicas.

En dicho lenguaje la unidad básica son las clases, que estarán compuestas por variables. Pueden especializarse de acuerdo a la función que se les desea asignar. Una de dichas especializaciones es la clase *conector*, que permite la interacción entre distintos objetos (básicamente posibilitan la introducción de restricciones, agregando ecuaciones que deben satisfacerse). Por otro lado, existe un modificador de variables, *flow*, que también posibilita la introducción de otro tipo de restricciones.

Al usar el operador *connect* con dos conectores, se introducen las siguientes restricciones:

- Las variables de flow de ambos conectores deben sumar cero.
- Todas las demás variables deben ser iguales.

A continuación se presenta un ejemplo, a modo de ilustración. Contamos con el conector Pin, incluido en la clase A, de la cual hay tres instancias (a, b, y c) que serán conectadas entre sí. Este es el código Modelica:

```
connector Pin
  Modelica.Units.SI.Voltage v;
  flow Modelica.Units.SI.Current i;
end Pin;

model A
  Pin p;
end A;

model Composition
  A a;
```

```

A b;
A c;
equation
  connect(a.p, b.p);
  connect(a.p, c.p);
end Composition;

```

Luego, las ecuaciones resultantes obtenidas al aplanar ambos connect son:

```

a.p.v = b.p.v;
a.p.v = c.p.v;
a.p.i + b.p.i + c.p.i = 0;

```

Entonces es necesario conocer qué conectores se encuentran conectados entre sí. Podemos pensar esto como un problema de grafos:

- Cada conector es representado mediante un nodo.
- Por cada connect, agregamos una arista entre los nodos que se corresponden con los conectores conectados.
- El objetivo es hallar el conjunto de *componentes conexas* del grafo. Una componente conexa es un conjunto de vértices del grafo, en el que se cumple que, para cualquier par de vértices del mismo, existe un camino que une ambos vértices.

Luego, es sencillo generar las ecuaciones correspondientes para cada componente conexa.

3.6. Modelica y sistemas de gran escala

Actualmente, debido al incremento experimentado en la capacidad de cómputo, se ha intentado modelar y simular sistemas de gran escala. Si bien Modelica permite modelar estos sistemas de manera sencilla, el proceso de conversión de EDAs a EDOs se torna inmanejable para sistemas de gran escala [2]. Ninguna de las herramientas es capaz de compilar modelos que trabajen con miles de variables.

El grupo que desarrolla ModelicaCC, a la par del desarrollo del compilador, ha estado trabajando en una teoría de *grafos basados en conjuntos* [10]. Dichos grafos tienen por vértices a conjuntos de elementos, y del mismo modo, las aristas se componen de conjuntos de pares de vértices. Si los mencionados conjuntos pueden ser descriptos de manera intensiva, luego las operaciones sobre dichos conjuntos tienen costo constante.

Esto puede resultar muy restrictivo para muchas aplicaciones, pero en Modelica los sistemas de gran escala suelen presentar regularidades, que permiten trabajar bajo estas condiciones.

Mariano Botta trabajó en la etapa de aplanado de sistemas de gran escalas, con un algoritmo que analiza la forma de las conexiones para obtener un resultado [1]. Pablo Zimmerman implementó los algoritmos de matching máximo y Tarjan para *grafos vectorizados* (una especialización de grafos basados en conjuntos, que no contemplan el caso multidimensional) [9]. Además, Franco Constantini trabajó en el problema de reducción de índices para el caso escalar,

presentando algunos conceptos básicos para el desarrollo en el caso vectorial, aunque sin formalizarlos en extremo [4]. Para el algoritmo de tearing, no se han hecho avances para este tipo de grafos particulares.

Hasta donde llegan los conocimientos de este grupo, no hay antecedentes en el trabajo de este problema.

4. Objetivos

4.1. Teoría de grafos basados en conjuntos

En primer lugar se desea estudiar y comprender la teoría de grafos basados en conjuntos. Dado que hay varios algoritmos ya propuestos, se tomarán los mismos como punto de partida, para generalizarlos.

4.2. Reemplazo de conexiones por ecuaciones

Se desea trabajar en el estudio de un algoritmo que utilice grafos basados en conjuntos para resolver el problema reemplazar conexiones por ecuaciones en la etapa de aplanado.

4.3. Generalización de algoritmos de matching y Tarjan para grafos basados en conjuntos

Del mismo modo, se desea utilizar a los grafos basados en conjuntos para la etapa de causalización.

4.4. Aplicación

Se implementarán las formalizaciones en el compilador de Modelica, ModelicaCC. Luego de hacerlo, se analizarán y compararán los resultados obtenidos mediante el uso de grafos basados en conjuntos, con respecto al caso tradicional, en sistemas de gran escala.

Habiendo presentado este marco teórico novedoso, algunos objetivos deseables serían los siguientes:

- Algoritmo de reducción de índices. Describir de manera formal un algoritmo de reducción de índices para grafos basados en conjuntos, basado en el algoritmo de Pantelides para el caso escalar.
- Algoritmo de tearing. Formalizar un algoritmo de tearing que trabaje con grafos basados en conjuntos.

Sin embargo el tiempo requerido para poder cumplirlos probablemente exceda el estipulado para este trabajo.

5. Plan de trabajo

El plan propuesto contempla una duración más de cuatro meses con una carga horaria de 4hs diarias. La fecha de inicio es el 15/06/2020, planificando su finalización para 15/11/2020.

Las tareas a realizar son:

1. Estudio de la teoría de grafos basados en conjuntos. Familiarización con ModelicaCC. Tiempo estipulado: un mes.
2. Implementación de grafos basados en conjuntos, y algoritmo de componentes conexas. Tiempo estipulado: un mes.
3. Estudio de la complejidad tempo-espacial de dichas implementaciones. Tiempo estipulado: una semana.
4. Estudio de algoritmo de Tarjan para grafos basados en conjuntos. Implementación en ModelicaCC. Tiempo estipulado: seis semanas.
5. Estudio de la complejidad tempo-espacial del algoritmo para componentes fuertemente conexas (Tarjan). Tiempo estipulado: una semana.
6. Escritura del informe. Tiempo estipulado: un mes.

6. Bibliografía básica

La bibliografía básica es aquella que se ha citado a lo largo de la propuesta, detallada en las referencias, al final del documento.

Referencias

- [1] Mariano Botta. “Aplanado eficiente de grandes modelos Modelica”. Tesis de mtría. FCEIA, 2015.
- [2] Francesco Casella. “Simulation of Large-Scale Models in Modelica: State of the Art and Future Perspectives”. En: sep. de 2015, págs. 459-468.
- [3] Franois Cellier y Ernesto Kofman. *Continuous System Simulation*. New York: Springer, 2006.
- [4] Franco Constantini. “Reducción de índices en grandes modelos Modelica”. Tesis de mtría. FCEIA, 2018.
- [5] Peter Fritzson y Vadim Engelson. “Modelica - A Unified Object-Oriented Language for System Modelling and Simulation”. En: *ECOOOP*. 1998, págs. 67-90.
- [6] Constantinos C. Pantelides. “The Consistent Initialization of Differential-Algebraic Systems”. En: *SIAM Journal on Scientific and Statistical Computing* 9.2 (1988), págs. 213-231.
- [7] L. R. Petzold. “A description of DASSL: a differential/algebraic system solver”. En: *Scientific computing (Montreal, Quebec, 1982)*. New Brunswick, NJ: IMACS, 1983, págs. 65-68.
- [8] Robert Tarjan. “Depth-First Search and Linear Graph Algorithms”. En: *SIAM Journal on Computing* 1.2 (1972), págs. 146-160. DOI: 10.1137/0201010.
- [9] Pablo Zimmermann. “Resolución de lazos algebraicos en grandes modelos Modelica”. Tesis de mtría. FCEIA, 2018.

- [10] Pablo Zimmermann, Joaquín Fernández y Ernesto Kofman. “Set-Based Graph Methods for Fast Equation Sorting in Large DAE Systems”. En: *Proceedings of EOOLT 2019* (2019).